

Pathfinder as a state machine

Pathfinder, github.com/vd1/pathfinder

12 September 2026, revised 13 September

What, why, how

What. Pathfinder takes two corpora of papers, Q and P, and looks for pairs (q, p) with something publishable between them: a result that neither paper states and that follows from putting the two together. For every pair it keeps, it produces a record in increasing order of readability and decreasing dependence on the process that made it: the *thread* (a ledger of everything the agents said to each other), the *note* (what the agents agreed and a judge accepted), the *readable note* (the same, rewritten for a reader who never saw the ledger), and, when the judge said the result would carry a paper, the *paper* (written with a literature search and reviewed).

Why a state machine. The work is done by model calls that cost money, take minutes, fail, and disagree. Writing the pipeline as a machine with explicit states, transitions and budgets is what makes it re-implementable by a colleague on another corpus, auditable after the fact (every decision traces to a file and a receipt), and safe to interrupt: a stop, a crash or an exhausted budget leaves the machine in a named state from which it resumes, and a human can see at a glance where each pair stands and what would move it.

How. One research thread is one instance of the machine described below. Its subject is a pair of papers together with the seed of an idea: in Pathfinder the seed is the scan's connexion, rationale and scores, handed to the peers in their first prompt; in the agQSL instance of the same machine the seed is a written idea artefact. The machine does not care where the seed came from, only that the thread starts from a pair and a seed. The campaign is a set of these instances plus a scheduler with a purse. Three kinds of actor appear: *agents*, model calls with a prompt and a bounded view of the state, of which the two *judges* (the verifier of a note, the reviewer of a paper) make the only decisions; the *pipeline*, code that builds, checks, records and moves the position; and the *owner*, whoever types in the terminal, a human or a delegated session, who raises budgets, edits by hand and settles what is paused. The agents of one thread share one append-only ledger; that ledger, the files beside it and a position record are the whole state of the thread. Everything below is what the code does; the builder prompt points here for the shape to re-implement.

1 Global state of a thread

Position. `stage`, where the machine is; `status`, one of `running`, `stopped`, `BLOCKED` or a terminal word; the counters `round` (research rounds used), `repairs` (note repairs used), the paper's `round` and the editor's `attempt`.

Memory, all files under `threads/<pair>/`:

- `inputs/`: the two papers, flattened TeX or PDF text, and their metadata; the seed, the pair's row of `scan.jsonl`, is read from the campaign level.

- `ledger.jsonl`: append-only attributed entries of kind idea, finding, objection, correction, intention, ready or review; the peers' shared memory and the verifier's evidence.
- one directory per peer (`ada/`, `emmy/`, ...): that peer's derivations, scripts and checks. The peers are a campaign setting (`peers`); two by default, three were tested.
- `<pair>.tex`: the consolidated note; `<pair>.verdict.json`: every verdict with the digest of the note it judged.
- `paper/`: the paper, its bibliography, its search record, the pipeline's checks per round, and every review with the digest of the paper it judged.
- `edited/`: the readable note (`note.tex`, `references.bib`, `note.pdf`).
- `status.json`, `paper/paper.json`, `edited/edit.json`: the position of each stage; `lock`: the pid of the runner working the thread.

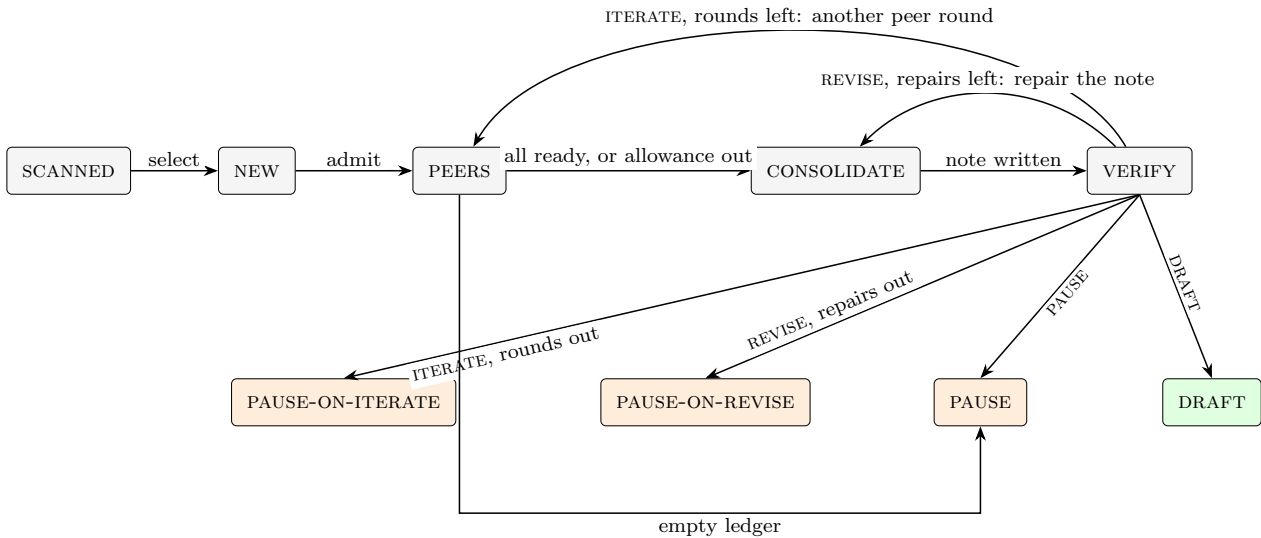
Campaign level. `fetch.json` (the two queries and how far each side has been paged), `Q.jsonl` and `P.jsonl`, `scan.jsonl`, `shortlist.json`, `receipts.jsonl` (the only spend figure), and the markers `stop.json` and `health.json`.

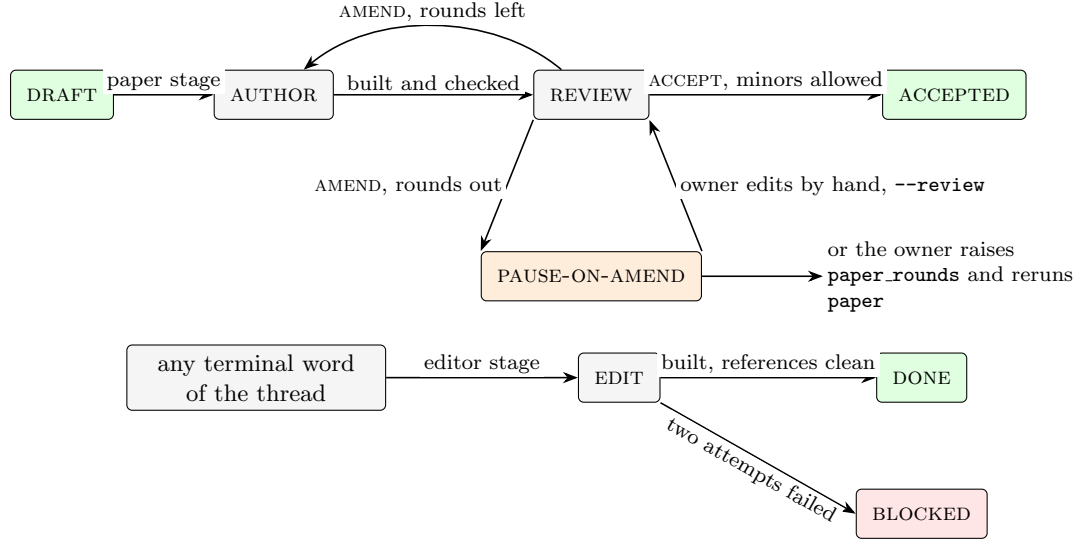
2 The ledger

The ledger is the one piece of state several agents write. Its rules are what make the peers a group rather than a set of monologues:

- Entries are appended through a helper, never edited; each carries a sequence number, an actor, a kind and a time. Peers read the whole ledger before every call.
- *ready* is a claim about the ledger as it stands: it is void as soon as anyone adds a substantive entry (anything but ready or review) after it. A peer whose ready still stands is not called again in that round; it waits for its partners.
- The round ends when every peer's ready stands, or when the round's allowance is out.
- The verifier writes its verdict into the ledger as a review entry, so the next round's peers, and the consolidator on a repair, read the judge's reasons where they read everything else.

3 States





Terminal words of the research thread: DRAFT, PAUSE, PAUSE-ON-ITERATE, PAUSE-ON-REVISE. Terminal words of the paper: ACCEPTED, PAUSE-ON-AMEND. Endings of the editor: DONE, BLOCKED. Every PAUSE-ON-X means “the loop named X ran out of its budget with the judge still asking”; the owner can raise the budget and resume from the recorded stage, or, for the paper, edit it by hand and ask for one more review. The editor runs once for every terminal thread, whatever its ending, so that even a paused pair leaves a readable account.

stopped and **BLOCKED** are not endings. **stopped** is a drain checkpoint (a stop marker, a transport failure, a kill) and resumes at the recorded stage on the next start. **BLOCKED** is an unreadable decision or a missing output: the reply is saved beside the position (for the verifier, `verify-unreadable.txt`), the runner never re-admits the pair, and **reconcile** names the one safe action.

stage	decisions of its judge	endings
research thread	DRAFT, REVISE, ITERATE, PAUSE	DRAFT, PAUSE, PAUSE-ON-ITERATE, PAUSE-ON-REVISE
paper	ACCEPT, AMEND	ACCEPTED, PAUSE-ON-AMEND, blocked
editor	none	DONE, blocked

4 Transitions

Each transition is one or more model calls, or a mechanical step by the pipeline or the owner. “Sees” is the part of the global state the agent can read; “writes” is what it may change. Nothing else is visible or writable, which is what makes the record auditable.

Research thread

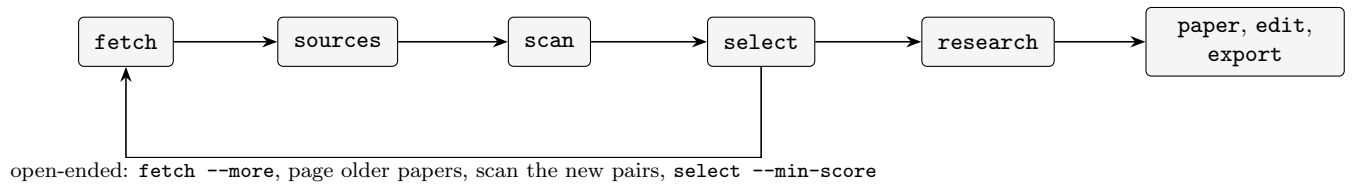
from → to	agent, prompt, tools	sees	writes	budget
corpus → SCANNED	judge, scan.md , no tools	Q and P titles and abstracts, or full text	one row of scan.jsonl	one call, retried once if unparseable
SCANNED → NEW	none (select)	all scan rows	shortlist.json : the top cut, or every pair at or above a threshold; started pairs are kept	none
NEW → PEERS	none (runner)	shortlist, statuses, locks, receipts, markers	lock, status.json	seats; guard: spend plus in-flight estimate under budget_usd , else the stop marker
PEERS → PEERS	every peer concurrently, peer.md , tools and web search	inputs, ledger, own and partners' directories, the scan's connexion and scores, this call's allowance	ledger entries through the helper, own directory	peer_calls per peer and peer_seconds shared, per round; a peer whose ready stands waits instead of calling
PEERS → CONSOLIDATE	none	ledger readiness	status.json	when every peer is ready or the allowance is out; an empty ledger goes to PAUSE
CONSOLIDATE → VERIFY	first peer, consolidate.md , tools	ledger, all directories, the prior note; on a repair also the verifier's corrections	<pair>.tex ; after an ITERATE it appends to the prior note, on a repair it rewrites	consolidate_seconds ; rerun once unless the note exists
VERIFY → next	verifier, verify.md , no tools	inputs, ledger, note, all inline	verdict file with the note's digest, a review entry in the ledger, status.json	verify_seconds ; rerun once on an empty reply; an unreadable verdict is saved and the pair is BLOCKED
VERIFY → PEERS	verdict ITERATE		round +1	rounds (4); at the cap PAUSE-ON-ITERATE
VERIFY → CONSOLIDATE	verdict REVISE	the consolidator also sees the corrections	repairs +1	repairs (1); at the cap PAUSE-ON-REVISE

Paper, owner and editor

from → to	agent, prompt, tools	sees	writes	budget
DRAFT → REVIEW	author, author.md , tools and web search	everything in the thread, the web, the previous review's findings if any	paper/: paper.tex , references.bib , search.md	paper_seconds per call
REVIEW → next	reviewer, review.md , no tools	paper, bib, the pipeline's build and reference checks, the search record, note, ledger, inputs, all inline	paper/review.json with the paper's digest, paper.json	review_seconds ; paper_rounds (3); at the cap PAUSE-ON-AMEND. ACCEPT may carry minor findings; AMEND only for a finding that changes a claim, its scope, attribution or correctness
PAUSE-ON-AMEND → REVIEW	the owner edits the paper by hand, then paper PAIR --review	the paper as it stands	one more review, counted as the next round	none: an owner round is outside the budget
PAUSE-ON-X → its loop	the owner raises the budget and reruns research or paper	the position	the loop continues from the recorded stage	the raised cap
terminal → EDIT	editor, editor.md , tools, no search	inputs and metadata, note, verdicts, ledger, all directories; on retry, the build log and the reference findings	edited/	edit_seconds ; two attempts, then blocked

Between transitions the pipeline itself, with no model, does the checks that gate the next state: it builds the $\text{\LaTeX}$, verifies that every citation exists and is used and that arXiv identifiers carry their titles, refuses an edited note whose references do not check, records digests, and writes the position. The verifier's and reviewer's judgements are the only decisions; every other arrow is mechanical or the owner's.

5 The campaign



scan.jsonl is append-only, so a longer corpus only adds rows; a threshold cut re-selects over all rows and keeps every pair that has a thread. **explore** runs the loop **fetch --more**, **scan**, **select --min-score** for a number of passes. The runner then admits whatever is NEW on the shortlist, so research can run while the corpus grows.

6 Budgets, and what exhausting them means

budget	scope	when exhausted
<code>peer_calls</code> , <code>peer_seconds</code>	one peer round	consolidate on the ledger as it stands
<code>rounds</code>	the ITERATE loop	PAUSE-ON-ITERATE
<code>repairs</code>	the REVISE loop	PAUSE-ON-REVISE
<code>paper_rounds</code>	the AMEND loop	PAUSE-ON-AMEND
editor attempts	the build and reference checks	blocked
<code>budget_usd</code>	the campaign, at admission	stop marker; threads in flight finish; NEW pairs wait

Every model call appends a receipt, so the campaign budget is checked against the sum of receipts, never against an estimate alone. A call that times out with no usage is charged at `call_estimate_usd`. Owner rounds are not budgeted: the owner is the budget.

7 Interrupts and hazards

- A stop marker (operator, guard, or first Ctrl-C) stops admission; calls in flight land and write their checkpoints; the runner exits; the next start resumes each pair at its recorded stage.
- No session within the grace time is a transport failure: no receipt, the pair is **stopped**, two in a row set the health marker and pause admission until a probe succeeds. The **paper** and **edit** commands skip the failed pair and continue.
- **reconcile** reads the position of one pair and names the one safe action: start, resume peers, run consolidate, run verify, or nothing; **--apply** performs it.
- The lock protects a thread from two runners. It does not protect the paper or editor stage from two concurrent commands on the same pair; that is an operator error the code does not yet catch.